

FunctorFlow v0: Diagrammatic Backpropagation with Kan Extensions

Sridhar Mahadevan

March 28, 2026

Textbook Reference: Mahadevan, Categories for AGI, 2026.

Download link: <https://people.cs.umass.edu/~mahadeva/papers/catagi.pdf>

Why do we need another language for deep learning?

- Modern deep learning is rich in constructions, but poor in unifying abstractions.
- We repeatedly build the same patterns under different names:
 - attention and aggregation,
 - message passing and neighborhood integration,
 - multimodal fusion and synchronization,
 - denoising, completion, and consistency repair.
- The field has accumulated many powerful mechanisms, but no common calculus for composing them.

Main thesis

Deep learning has reached the point where abstraction is no longer optional: it is the only way to streamline recurring constructions.

How category theory emerged in mathematics

- Category theory did not appear out of nowhere.
- Mathematicians kept rediscovering the same constructions in many settings:
 - products of groups, rings, and topological spaces,
 - unions, quotients, and gluings,
 - structure-preserving maps between algebraic and geometric objects.
- Over time, these repeated patterns were compressed into a small language:

products $\rightarrow$ limits, unions/gluings $\rightarrow$ colimits, mappings $\rightarrow$ functors.

Historical lesson

Category theory did not add abstraction for its own sake. It removed accidental complexity by naming recurring structure.

Deep learning is now at a similar moment

Before abstraction in math

- products everywhere
- quotients everywhere
- maps everywhere
- repeated proofs, repeated constructions

Before abstraction in deep learning

- attention everywhere
- message passing everywhere
- diffusion and denoising everywhere
- fusion, planning, and consistency everywhere

Observation

We are repeating the same compositional patterns across architectures, but we still describe them as disconnected inventions.

FunctorFlow: a working unifying layer

Idea

FunctorFlow is now a working v0 language for specifying deep learning architectures as diagrams of typed objects and morphisms, then compiling those diagrams into executable models.

Diagram / Spec $\longrightarrow$ Categorical IR $\longrightarrow$ Neural Architecture

- **Objects** represent data types, states, or latent spaces.
- **Morphisms** represent transformations, layers, or learned maps.
- **Diagrams** represent whole architectures.
- **Universal constructions** give reusable architectural primitives.

The compiler analogy

Classical compilers

- high-level program
- typed intermediate representation
- machine code
- semantics preserved by construction

FunctorFlow

- high-level diagram
- categorical intermediate representation
- neural executable
- structure preserved by construction

Separate verifiable compilation from run-time behavior

We aim to compile *structure*, localize uncertainty to learned boundary components, and enforce global consistency internally.

A first translation dictionary

Category theory	Deep learning intuition
Object	representation space / data type
Morphism	layer / transformation / update rule
Diagram	architecture / computation graph
Functor	representation-changing pipeline
Left Kan extension	aggregation / attention / message integration
Right Kan extension	completion / denoising / repair
Pullback	synchronization / alignment
Sheaf gluing	multi-view consistency

Takeaway

FunctorFlow is not a metaphor. It names recurring deep learning constructions explicitly, compositionally, and reusably.

See Categories for AGI for Example Systems implementable in FunctorFlow

- **KET**: left Kan aggregation as a universal attention-like primitive.
- **DB**: commutativity violations as structural obstruction signals.
- **GT**: message passing over simplicial or neighborhood diagrams.
- **BASKET / ROCKET**: planning and local search in compositional diagram spaces.
- **Democritus**: sheaf-style gluing and consistency across views.

Reframing

These are not disconnected ideas. They are instances of a single compositional calculus.

Roadmap for this tutorial

- ➊ Why deep learning needs a unifying compositional language.
- ➋ The core primitives of FunctorFlow.
- ➌ Kan extensions as architectural building blocks.
- ➍ Reinterpreting KET, DB, GT, BASKET, and ROCKET inside one framework.
- ➎ The current FunctorFlow v0 implementation, notebook demos, and proof interface.
- ➏ Toward consistency-aware and partially verifiable deep learning systems.

From a collection of models to a language of constructions.

What exists today in FunctorFlow v0

- A working package-level DSL inside FunctorFlow/.
- Core layers: typed diagrams, categorical IR, and Torch lowering.
- Reusable macro libraries for KET, DB, GT, BASKET, ROCKET, and Democritus.
- Ports, typed adapters, tutorial libraries, and generated notebooks.
- Real PTB and Wiki-2 language-model demos, including Transformer, GT, and KET comparisons.
- A synthetic predict-detach regime demo for causal vs. leaky aggregation behavior.
- A Lean proof-of-concept interface for compilation certificates.
- Package-local data under FunctorFlow/data/ for GitHub portability.

Practical status

FunctorFlow is an evolving language, and will be more fully presented at ICML 2026 in July

Why start with left Kan extensions?

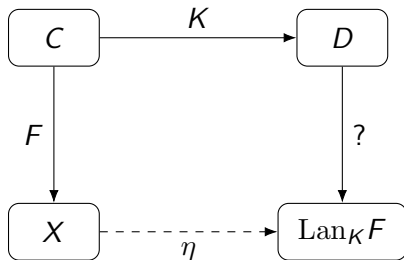
- Aggregation is one of the most repeated operations in modern deep learning.
- We see it under many names:
 - attention,
 - pooling,
 - neighborhood message passing,
 - patch aggregation,
 - plan-fragment integration.
- These look different operationally, but often share the same structural role:

local data $\longrightarrow$ global summary consistent with a map of contexts.

Thesis

Left Kan extensions give a universal language for aggregation.

The conceptual picture



- We have a description F on a category of local contexts C .
- We want the *best extension* of that description along a map $K : C \rightarrow D$.
- The left Kan extension is the universal way to push local information forward.

The formal slogan

For a functor $F : C \rightarrow \mathcal{E}$ and an indexing map $K : C \rightarrow D$, the left Kan extension

$$\mathrm{Lan}_K F : D \rightarrow \mathcal{E}$$

is the universal object satisfying

$$\mathrm{Nat}(\mathrm{Lan}_K F, G) \cong \mathrm{Nat}(F, G \circ K)$$

for every $G : D \rightarrow \mathcal{E}$.

Interpretation

Instead of inventing an aggregation rule ad hoc, we ask for the universal way to summarize local information on C as a construction on D .

ML translation of the universal property

Category theory

- C : local neighborhoods / patches / fragments
- D : target positions / tokens / global states
- K : incidence or neighborhood map
- F : local values or messages
- $\text{Lan}_K F$: aggregated representation on D

Deep learning

- gather context
- align it to a target index
- aggregate consistently
- produce a contextualized output

Key message

Left Kan says: aggregation should be determined by structure, not just by a hand-designed tensor trick.

Attention as a special case of left Kan

- In self-attention, each token position gathers information from a context of source positions.
- Queries, keys, and values define a context-selection mechanism.
- The final weighted sum is an aggregation indexed by a neighborhood relation.

tokens $\xrightarrow{\text{incidence / relevance}}$ contexts $\xrightarrow{\text{aggregate}}$ contextualized tokens

FunctorFlow view

Self-attention is one concrete realization of a more general left Kan aggregation pattern.

From attention to KET

- Standard attention uses a dense relevance relation over tokens.
- KET generalizes this by treating aggregation as a left Kan extension along a chosen incidence structure.
- The incidence structure can encode:
 - sequence neighborhoods,
 - geometric adjacency,
 - simplicial structure,
 - plan-fragment relations,
 - topic or causal neighborhoods.

Upgrade

KET is not merely another attention variant. It is a universal aggregation template whose concrete realizations recover many known architectures.

The FunctorFlow programming promise

User-facing idea

A left Kan block should be specified at the level of diagrams, not raw tensor plumbing.

```
cfg = KETBlockConfig(name="TutorialKET", ...)
D = ket_block(cfg)
model = compile_to_torch(
    D,
    reducers={"ket_attention": ket_reducer}
)
```

- The macro captures the categorical structure.
- The compiler lowers it to executable tensor code.
- The same API already underlies the current PTB and Wiki-2 KET demos.

Summary of Main Points

- ① Aggregation is everywhere in deep learning.
- ② Left Kan extensions provide a universal description of aggregation.
- ③ Attention is one special case of this broader construction.
- ④ KET becomes the first canonical FunctorFlow primitive.

From tensor tricks to universal constructions.

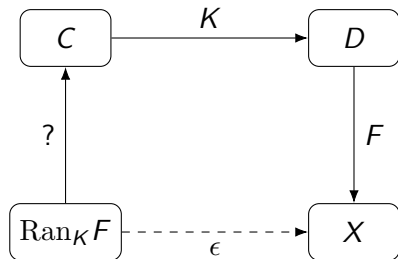
Why right Kan extensions belong next to left Kan

- If left Kan extensions express aggregation, right Kan extensions express the dual operation: *completion under constraints*.
- In deep learning, this pattern also appears everywhere:
 - denoising and repair,
 - masked prediction,
 - diffusion-style completion,
 - filling in missing plan fragments,
 - reconciling partial views into a consistent whole.
- If left Kan is about pushing local evidence forward, right Kan is about inferring what must hold globally to remain consistent with that evidence.

Thesis

Right Kan extensions give a universal language for completion, denoising, and repair.

The dual conceptual picture



- We have a target description F defined on a category D .
- We want the *best completion* viewed back through a map $K : C \rightarrow D$.
- The right Kan extension is the universal way to infer missing structure while respecting constraints.

The formal slogan

For a functor $F : C \rightarrow \mathcal{E}$ and an indexing map $K : C \rightarrow D$, the right Kan extension

$$\mathrm{Ran}_K F : D \rightarrow \mathcal{E}$$

is the universal object satisfying

$$\mathrm{Nat}(G, \mathrm{Ran}_K F) \cong \mathrm{Nat}(G \circ K, F)$$

for every $G : D \rightarrow \mathcal{E}$.

Interpretation

Instead of inventing a repair rule ad hoc, we ask for the universal way to complete a partially specified structure so that it remains compatible with the observations we already have.

ML translation of the universal property

Category theory

- C : observed fragments / visible contexts
- D : latent global structure to be inferred
- K : observation or projection map
- F : partial information on what is observed
- $\text{Ran}_K F$: most consistent completion on D

Deep learning

- observe a noisy or incomplete view
- infer missing pieces
- enforce compatibility across contexts
- return a repaired representation

Key message

Right Kan says: completion should be determined by consistency constraints, not just by unconstrained generation.

From denoising to right Kan intuition

- In denoising, we start from a corrupted or partial object.
- The task is not merely to generate something plausible.
- The task is to reconstruct something *compatible* with the available evidence and with the structural relations encoded by the model.

partial observations $\xrightarrow{\text{compatibility / constraints}}$ repaired global object

FunctorFlow view

Denoising, masked completion, and repair can be treated as concrete realizations of a more general right Kan pattern.

Left Kan and right Kan: the symmetry

	Left Kan	Right Kan
Intuition	aggregate local evidence	complete from partial evidence
ML role	attention / pooling / message passing	denoising / repair / masked inference
Direction	local $\rightarrow$ global summary	partial $\rightarrow$ globally compatible completion
Tagline	universal aggregation	universal completion

Design principle

FunctorFlow gets its first major symmetry from pairing left Kan extensions for aggregation with right Kan extensions for repair.

Why this matters for hallucination and safety

- Unconstrained generation can produce outputs that are fluent but structurally inconsistent.
- A right Kan perspective reframes generation as *completion under compatibility constraints*.
- This does not guarantee truth, but it can:
 - localize uncertainty,
 - measure inconsistency,
 - guide repair toward globally compatible outputs.

Hallucination detection is a desirable but not yet implemented goal

We are not promising hallucination-free models. We are proposing architectures in which inconsistency becomes structurally visible and completion becomes constraint-aware.

From right Kan to our systems

- **KET**: left Kan for aggregation, with right Kan as the natural dual for completion.
- **DB**: obstruction signals tell us when different completion paths disagree.
- **Democritus**: reconcile partial topic-level views into a coherent relational manifold.
- **ROCKET**: repair candidate plans by searching for more globally compatible local edits.

Reframing

Right Kan gives the missing half of the story: once we know how to aggregate information, we also need a principled way to repair and complete it.

The FunctorFlow programming promise, part II

User-facing idea

A right Kan block should look as natural in code as a left Kan block.

```
cfg = ROCKETRepairConfig(name="RepairBlock", ...)
D = rocket_repair_block(cfg)
model = compile_to_callable(D)
```

- The user specifies the completion structure.
- The compiler lowers it to a denoising or repair module.
- The same abstraction can underlie masked modeling, diffusion-style repair, and consistency-aware inference.

Main Summary: Left vs. Right Kan

- ① Left Kan extensions provide a universal language for aggregation.
- ② Right Kan extensions provide a universal language for completion and repair.
- ③ Together they give FunctorFlow its first major architectural symmetry.
- ④ This symmetry connects attention, denoising, planning, and consistency-aware inference inside one framework.

From universal aggregation to universal completion.

Hallucination as structural obstruction

- Standard language models often fail in a familiar way: the output is fluent, but incompatible with the underlying evidence.
- We propose to view these failures not only as probabilistic errors, but as *structural obstructions*.
- The key question is not just “is the output likely?”
- The key question is:

does the output fit into a globally consistent diagram?

FunctorFlow stance

Hallucination is not merely low-probability text generation gone wrong. It is the failure to extend partial observations into a coherent global structure.

The compiler contrast

Classical compiler

- input program has fixed syntax and semantics
- typing and structure are explicit
- compilation preserves meaning by construction
- no room for hallucination

Unconstrained generation

- input prompt underdetermines the output
- structure is mostly implicit
- model fills gaps statistically
- inconsistency can slip through unnoticed

Goal

We cannot make deep learning identical to compilation, but we can make more of it *compiler-like*: explicit structure, typed composition, and checkable consistency conditions.

A Pearl-style separation of structure and uncertainty

- In structural causal models, the internal equations are deterministic.
- Uncertainty is localized in exogenous variables and imperfect observations.
- We adopt the same design philosophy for FunctorFlow.

Inside the structural core

- objects and morphisms
- compositional diagrams
- Kan constructions
- gluing and synchronization constraints

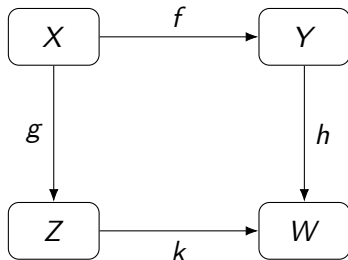
At the uncertain boundary

- noisy inputs
- learned embeddings
- partial observations
- stochastic approximation

Design principle

Localize uncertainty at the boundary; enforce deterministic compositional structure internally.

When a diagram should commute but does not



- Two paths represent two ways of deriving the same target claim.
- In an ideal system, the square commutes:

$$h \circ f = k \circ g.$$

- In a hallucinating system, these two paths disagree.

Interpretation

A hallucination is a witness that a supposedly consistent square does not actually commute.

From commutators to obstruction signals

Suppose two routes through the architecture should agree on a representation of the same object. Then a natural obstruction signal is

$$\mathcal{O}(x) = \|T_1(T_2(x)) - T_2(T_1(x))\|.$$

- If $\mathcal{O}(x) \approx 0$, the two routes are consistent.
- If $\mathcal{O}(x)$ is large, the model is structurally disagreeing with itself.
- This turns hallucination into something *observable* rather than purely qualitative.

DB viewpoint

Diagrammatic backpropagation promotes these obstruction signals from diagnostics to trainable structural losses.

Sheaf intuition: local agreement vs global inconsistency

- A model may look correct on each local fragment while still failing to glue into a coherent global story.
- This is exactly the kind of phenomenon sheaf theory was built to describe.
- Local sections may exist, but a global section may be obstructed.

local plausibility $\neq$ global consistency

FunctorFlow interpretation

Hallucination can be reframed as a failure of gluing: the model produces plausible local pieces that cannot be assembled into a single consistent object.

How left and right Kan fit the picture

Left Kan

- aggregates local evidence
- builds contextual summaries
- can expose conflict between neighborhoods

Right Kan

- completes from partial evidence
- repairs missing or noisy structure
- projects candidates back into a more compatible space

Combined picture

Hallucination becomes a mismatch between aggregation and completion: the system cannot turn its local evidence into a globally consistent repaired object.

The FunctorFlow programming promise, part III

User-facing idea

Structural consistency should be expressible at the level of diagrams and losses.

```
D = Diagram("ConsistencyAwareLM")
planning = D.use_tutorial_library("planning")
stack = planning.build_macro("basket_rocket_pipeline")
loss = D.obstruction_loss(paths=[("p1", "p2")])
```

- aggregation is explicit,
- repair is explicit,
- inconsistency is explicit.

Hallucination and Structural Inconsistency

- ① Hallucination can be reframed as structural inconsistency.
- ② Non-commuting diagrams and failed gluings provide measurable obstruction signals.
- ③ Left Kan aggregates evidence; right Kan repairs partial structure.
- ④ FunctorFlow aims to make these constraints native architectural objects.

From fluent outputs to structurally coherent systems.

From vision back to concrete systems

Key message

FunctorFlow is a high-level language in which real-world systems can be designed.

- **DB**: diagrammatic consistency and commutativity diagnostics.
- **GT**: geometric message passing over simplicial neighborhoods.
- **BASKET**: agentic plan construction as compositional workflow diagrams.
- **ROCKET**: local search and repair in plan space.
- **Democritus**: sheaf-like gluing of local claims into a relational manifold.

What looked disconnected becomes a family of diagrams.

DB: Diagrammatic Backprop as commutativity control

Reinterpretation

DB can be understood as attaching *diagrammatic constraints* to neural computation and measuring the extent to which alternative paths fail to agree.

- Compose transformations in two different orders.
- Measure the gap between the resulting states.
- Use this gap as a training signal, diagnostic, or obstruction score.

$$\mathcal{O}_{\text{DB}}(x) = \| T_1(T_2(x)) - T_2(T_1(x)) \|.$$

FunctorFlow view

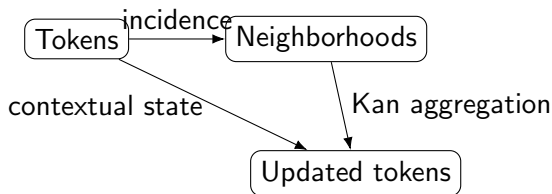
DB is the native language of **non-commuting diagrams**.

GT: Geometric Transformers as simplicial diagrams

Reinterpretation

GT augments standard token processing with neighborhood geometry, simplicial interactions, and topological diagnostics.

- tokens become objects,
- neighborhood and incidence maps become morphisms,
- message passing becomes structured aggregation along these morphisms,
- topological diagnostics become obstruction summaries over the diagram.



BASKET: Plans as compositional diagrams

Reinterpretation

BASKET turns textual evidence into workflows. In FunctorFlow, those workflows become explicit diagrams whose objects are plan states and whose morphisms are action transitions.

- extracted actions become typed morphisms,
- workflows become paths,
- plan libraries become reusable subdiagrams,
- composition becomes the central abstraction for agentic reasoning.

observe $\rightarrow$ analyze $\rightarrow$ optimize $\rightarrow$ realize_revenue

FunctorFlow view

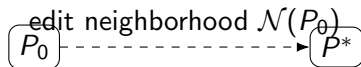
BASKET is not just sequence modeling; it is **diagram synthesis in plan space**.

ROCKET: Repair and local search in diagram space

Reinterpretation

ROCKET evaluates edits to a candidate plan and selects a structurally improved alternative according to a reward. In FunctorFlow language, this is local search over a neighborhood of diagrams.

- insert a morphism,
- delete a morphism,
- merge a reusable macro-skill,
- complete a partial path.



FunctorFlow view

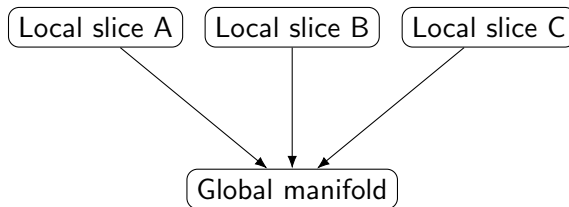
ROCKET is a **right-Kan-flavored repair story** in plan space.

Democritus: Sheaf-style gluing of local claims

Reinterpretation

Democritus extracts many local relational fragments and attempts to unify them into a coherent global manifold. This is exactly the intuition behind sheaf-style gluing.

- local claims are sections,
- overlapping topics define compatibility regions,
- failed agreement creates obstructions,
- successful unification yields a global relational state.



One framework, many instances

System	FunctorFlow interpretation	Key primitive
DB	commutativity diagnostics	obstruction
GT	simplicial message passing	left Kan + geometry
BASKET	workflow composition	diagram synthesis
ROCKET	plan repair / completion	local search + right Kan
Democritus	multi-view claim gluing	sheaf consistency

Takeaway

The point is not that these systems are identical. The point is that they can be expressed in a *shared calculus*.

How this looks in FunctorFlow

```
D = Diagram("CorporatePlan")
state = D.object("PlanState")
obs    = D.object("Observations")
frag   = D.object("Fragments")

agg     = D.left_kan(source=frag, along=obs)
rep     = D.right_kan(source=frag, along=state)
obstr   = D.obstruction_loss(paths=[("draft","repair")])
```

- the same API can express aggregation,
- repair,
- and inconsistency monitoring.

Why this matters

- ① It lowers the barrier to entry: one language instead of many disconnected stories.
- ② It clarifies what is reusable across applications.
- ③ It turns category theory from background philosophy into executable design.
- ④ It gives a path from tutorial intuition to software artifacts and formal semantics.

FunctorFlow is the bridge from theory to systems.

Working software artifacts today

- `core.py`: typed objects, morphisms, diagrams, ports, adapters.
- `compiler.py`: categorical IR execution and Torch lowering.
- `macros.py`: named blocks for KET, DB, GT, BASKET, ROCKET, Democritus.
- `ket_lm.py`: FunctorFlow-backed KET language models and presets.
- `lm_compare.py`: Transformer, GT, and KET comparison harnesses.
- `proof_interface.py` and `proofs/`: Lean certificate export and checking.
- `notebooks/`: generated PTB, Wiki-2, comparison, predict-detach, and tutorial notebooks.
- `data/`: package-local PTB and Wiki-2 datasets for standalone release.

Important software decision

All publishable tutorial code now lives inside `FunctorFlow/`, so the folder can become a standalone GitHub repository with minimal restructuring.

Notebook artifacts

- `ket_demo_functorflow.ipynb`
- PTB / Wiki-2 KET LM notebooks
- PTB / Wiki-2 model-comparison notebooks
- `predict_detach_regime_demo.ipynb`

What they demonstrate

- typed KET blocks
- Torch lowering through FunctorFlow
- historical-style KET presets
- reproducible PTB and Wiki-2 smoke runs
- side-by-side Transformer / GT / KET comparisons
- causal vs. leaky vs. predict-detach regime behavior

Interpretation

The tutorial now reaches real-dataset notebooks, comparative baselines, and a causal-structure sanity check.

Lean proof interface: what is actually verified?

Current proof-of-concept

- serialize a FunctorFlow diagram into a small Lean IR
- check that operation references are declared
- check that exposed ports resolve to declared refs
- check that lowered operations match declared operations

What this gives us

- proof-carrying compilation artifacts
- a checkable story about structural correctness
- a bridge from categorical design to formal semantics
- a credible path toward richer certified interfaces

Lean Verification Scope

We are not formally verifying learning performance. We are verifying a small but meaningful part of the compilation process itself.

Transition to implementation

Next question

If these ideas really belong to one calculus, what is the smallest software stack that makes them executable and publishable?

Next: FunctorFlow as an implemented deep learning compiler.

- We have a unifying language
- We have concrete model instances
- Now we need a way to build systems

FunctorFlow: A Deep Learning Compiler

Classical compilers:

- Program $\rightarrow$ Intermediate Representation $\rightarrow$ Machine code
- Enforce structure and correctness

FunctorFlow:

- Diagram $\rightarrow$ Categorical IR $\rightarrow$ Neural architecture
- Enforces compositional structure

We compile structure, not just parameters

- Objects: data types / representations
- Morphisms: transformations / layers
- Diagrams: model architectures
- Kan extensions: aggregation and completion

Models are diagrams

Minimal API

```
Tokens = Object("Tokens")
Nbrs    = Object("Nbrs")

inc = Morphism("incidence", Tokens, Nbrs)
val = Morphism("value", Nbrs, Nbrs)

D = Diagram("KET")
D.add(inc, val)

agg = D.left_kan(source=val, along=inc)

model = compile_to_torch(D)
```

Diagram $\rightarrow$ Model

What Gets Compiled

- Diagram structure
- Composition of morphisms
- Aggregation operators (Kan)
- Constraints and consistency structure

Structure is compiled, parameters are learned

What We Gain

- Compositional model design
- Reusable architectural primitives
- Explicit structure and constraints
- Measurable consistency (obstructions)

From ad hoc architectures to structured systems

- ① July 6, Seoul, South Korea.
- ② A more complete FunctorFlow implementation will be presented
- ③ **Textbook**: Mahadevan, Categories for AGI, 2026.
- ④ **Download link**: <https://people.cs.umass.edu/~mahadeva/papers/catagi.pdf>

Summary

FunctorFlow is both a conceptual unifier and a platform for efficiently designing structured deep learning systems.